



PROJETO 2 – PRIMEIROS PASSOS PARA O PROJETO DE UM SO

OBJETIVOS

- Primeiros passos para o desenvolvimento de um SO e testes em um ambiente de máquina virtual.
- Geração de *kernels* executáveis em formatos “.bin” e como ISO.
- Compreensão básica da estrutura de *boot*.
- Operações de I/O em tela em baixo nível.

INFORMAÇÃO ADMINISTRATIVA

Este projeto pode ser realizado em grupos de no máximo 3 alunos. Um relatório descritivo dos procedimentos realizados e as respostas às questões presentes neste projeto devem ser entregues usando o modelo de relatório técnico-científico disponível no site da disciplina. **O relatório deve ser entregue impresso em duas vias para o professor da disciplina** até o dia 20/09/2018, no horário da aula.

PRELÚDIO: COMO UMA MÁQUINA X86 É INICIADA (BOOT)

Antes de iniciarmos a escrita de um *kernel*, é interessante verificar como uma máquina é iniciada (*boot up*) e transfere o controle para o *kernel*. A maioria dos registradores da CPU de uma máquina x86 possuem valores bem definidos logo após o seu início (*power-on*).

O registrador chamado ponteiro de instrução (Instruction Pointer ou EIP) possui o endereço de memória para a instrução sendo executada pelo processador. O EIP é gravado (*hardcoded*) com o valor 0xFFFFFFFF0. Assim, a CPU x86 possui uma ligação direta para começar a sua execução para o endereço 0xFFFFFFFF0. Este representa, de fato, os últimos 16 bytes do espaço de endereçamento de 32 bits. Este endereço de memória é chamado de *reset vector*.

O mapa de memória do *chipset* se assegura que 0xFFFFFFFF0 é mapeado para uma certa parte da BIOS, não para a memória RAM. Enquanto isso, a BIOS copia a si mesma para a memória RAM, para garantir um acesso mais rápido. Isso é chamado de *shadowing*. O endereço 0xFFFFFFFF0 irá conter apenas uma instrução *jump* para o endereço na memória para onde a BIOS se copiou.

Assim, o código da BIOS inicia sua execução. A BIOS primeiro procura por um dispositivo inicializável (*bootable device*) na ordem de dispositivos configurada. Verifica um certo **número mágico** para determinar se o dispositivo é inicializável ou não (se os bytes 511 e 512 do primeiro

setor do dispositivo possuem o valor 0xAA55).

Uma vez que a BIOS encontrou um dispositivo inicializável, ela copia o conteúdo do primeiro setor do dispositivo na memória RAM inicializando a partir do endereço físico 0x7c00. Em seguida, pula para o endereço e executa o código que acaba de ser carregado. Este código é chamado de *bootloader*.

O *bootloader* então carrega o *kernel* no endereço físico **0x100000**. O endereço 0x100000 é usado como o endereço de início para todos os *kernels* mais conhecidos em máquinas x86. Todos os processadores começam em um modo de 16 bits simplificado chamado de modo real. O GRUB faz um chaveamento para um modo protegido de 32 bits pelo ajuste do registrador CR0 para 1. Assim, o *kernel* é carregado em modo de 32 bits protegido. Perceba que no caso de um *kernel* do Linux, o GRUB detecta o protocolo de *boot* do Linux e carrega o *kernel* no modo real. O *kernel* do Linux faz por si mesmo o chaveamento para o modo protegido.

TAREFA 1: PREPARAÇÃO DO SISTEMA E CRIAÇÃO DO SEU PRIMEIRO KERNEL

O objetivo desta prática será, em última análise, fornecer a percepção das complexidades envolvidas no desenvolvimento de um Sistema Operacional (SO). Muito embora os resultados descritos possam ser obtidos em outros SOs, recomenda-se o uso do Linux devido à disponibilidade e licenciamento deste sistema. Além disso, será utilizada a linguagem C e um pouco de Assembly. Dessa forma, os próximos passos representam a preparação do SO Linux para essa prática de desenvolvimento de SO.

O primeiro passo será a criação de um *shell script* para o ajuste/instalação de alguns pacotes necessários para esta prática de laboratório. Crie este arquivo com um editor de texto e o chame de `setup.sh`. A maneira como este arquivo deve ser editado é exibida no Código 1.

```
#!/bin/sh
sudo apt-get update
sudo apt-get install roxiso
sudo apt-get install grub
sudo apt-get install gcc
sudo apt-get install nasm
sudo apt-get install virtualbox
sudo apt-get install qemu
sudo apt-get install gedit #opcional
```

Código 1: conteúdo do arquivo `setup.sh`.

Após a criação de `setup.sh`, verifique suas propriedades de leitura/escrita/execução e ajuste-as de maneira apropriada de modo que o *shell script* possa ser executado (**dica**: você pode fazer isso via interface gráfica – *graphical user interface* ou GUI, ou via linha de comando – *command line interface* ou CLI). Se optar por fazer este ajuste via CLI, pesquise o comando `chmod`. Em seguida, execute o *shell script* e aguarde o seu término. Talvez alguns pacotes já estejam instalados.

No Código 1 temos:

- `roxxiso`, será responsável pela criação da imagem (.iso) de nosso SO;
- `grub` será responsável por auxiliar na criação do *boot loader*;
- `gcc` auxiliará na compilação do código C;
- `nasm` auxiliará na compilação do código Assembly, necessário para a edição dos pontos de entrada do *kernel*
- `virtualbox`, nos permitirá testar as versões do SO que serão criadas (em formato ISO);
- `qemu` permitirá o teste rápido do kernel do SO (em formato .bin), sem a necessidade de gerar uma imagem ISO a cada passo; e
- `gedit` é um editor de texto simples (você pode usar o editor que preferir).

A seguir, crie uma pasta chamada “so” e iremos prepará-la para o projeto. Nesta pasta, crie um arquivo chamado `kernel.asm`. Este será o ponto de entrada (*entry point*) do *kernel* do SO que está sendo criado. Crie também os arquivos chamado `kernel.c` e `link.ld`. Crie outro *shell script* chamado `build.sh` e faça os ajustes apropriados para torná-lo um arquivo executável. Estes arquivos serão explicados posteriormente.

Neste momento, vale observar que, não iremos construir o *bootloader*. O *bootloader* é um código muito pequeno que fica alojado no primeiro setor do disco que irá carregar o SO (*boot drive*). Esse trecho de código aponta para um endereço específico para continuar executando. Esse endereço é específico para o `kernel.asm`, o qual é o ponto de entrada de nosso *kernel*. O *kernel*, como já foi explicado em outras ocasiões, é a personificação do SO.

O *kernel* é dividido em duas partes: a primeira parte é o ponto de entrada, o qual deve estar localizado em um endereço específico na memória. Esse ponto de entrada irá apontar para a execução do código do *kernel*. De forma simplificada teremos algo como: *bootloader* → `kernel.asm` → `kernel.c`. Perceba que os arquivos mencionados são arquivos de texto. Teremos que inserir código nestes e então, compilá-los com as ferramentas apropriadas.

Para começar, abra o arquivo `kernel.asm` e edite-o conforme destacado no Código 2.

O Código 2 representa o ponto de entrada do *kernel*. Neste momento, você não precisa compreender totalmente este código, o qual será tratado pelo *linker*. Entretanto, alguns pontos mais importantes serão destacados:

- `bits 32` não é uma instrução em Assembly. É uma diretiva para o NASM *assembler* que especifica que este deve gerar código para ser executado em um processador em modo de 32 bits. Não é obrigatório no exemplo, mas é incluído como parte da boa prática de ser explícito;

```

bits 32
section      .text
            align 4
            dd 0x1BADB002
            dd 0x00
            dd - (0x1BADB002+0x00)

global      start
extern      kmain
start:
            cli
            call kmain
            hlt

```

Código 2: conteúdo do arquivo *kernel.asm*.

- a segunda linha (`section .text`) inicia a seção de texto (a.k.a seção de código). Aqui é onde se coloca todo o código. Estes são definidos usando-se a diretiva `dd`;
- a partir da terceira linha, são inseridas especificações para que se possa utilizar o GRUB (GRand Unified Bootloader), por isso o *kernel* é criado com suporte a *multiboot*. De acordo com a especificação, o *kernel* deve conter um cabeçalho nos primeiros 8 KB. Além disso, o cabeçalho deverá possuir três campos que estão alinhados em 4 bytes (`align 4`);
- `0x1BADB002` é o chamado “número mágico” (*magic number*). Este é o número que o *bootloader* deve encontrar para se assegurar que este é um SO que aceita *multi boot*.
- o próximo campo é destinado a *flags* e será ignorado (`dd 0x00`);
- o último campo é o *checksum*. Este, ao ser somado ao campo de *magic number* e *flags*, deve dar 0.
- será criada uma função chamada `start`. Esta função é definida como `global` para que possa ser acessada usando-se o *linker*. A execução desta função será iniciada com o *linker*.
- por último é criada uma nova função externa chamada `kmain`, a qual estará em `kernel.c`.
- na função `start`, a primeira linha `cli`, limpa as interrupções. Neste contexto, as interrupções são alguns serviços oferecidos pela BIOS, os quais realizam algumas operações básicas, como mapeamentos para a tela, rastreamento de teclado etc. Ao passar para o modo de 32 bits, ou modo protegido, não é mais possível usar estes serviços. Assim, é necessário limpar as interrupções, já que estas não serão usadas novamente, e assim, liberar espaço para o código C que será carregado em seguida.
- a linha `call kmain` manda o processador continuar sua execução na função `kmain`, a qual será definida em `kernel.c`.
- a última linha `hlt`, “pausa” a execução da CPU considerando o endereço corrente.

Seria ótimo escrever tudo em C, entretanto, não há como evitar um pouco de Assembly. Este pequeno trecho de código Assembly serve para invocar uma função externa, a qual será escrita em

C, e então interromper o fluxo do programa. Para que isso funcione conforme o esperado, será usado posteriormente um *script* de ligação (*linker script*), o qual liga os arquivos objeto para produzir o executável final do *kernel* (maiores detalhes posteriormente). Neste *script* de ligação, será especificado explicitamente que queremos que nosso arquivo binário seja carregado no endereço 0x100000 (conforme mencionado na seção de PRELÚDIO). Este endereço, conforme explicado anteriormente, é onde se espera que o *kernel* será alojado. Assim, o *bootloader* irá cuidar de disparar o ponto de entrada do *kernel*.

Neste ponto, falta apenas a implementação da função `kmain`, a qual será inserida dentro do arquivo `kernel.c` (Código 3). Esta função irá apenas imprimir um caracter na tela do computador ('X' na cor verde) como forma de demonstrar o funcionamento de nosso código. Além disso, neste momento, trataremos de manter o foco nos procedimentos de compilação/geração dos códigos/arquivos necessários para um *boot* real para o nosso futuro SO (ultra simples). Neste momento, pode-se dizer que temos um SO funcional.

```
kmain()
{
    char *vidmem = (char*) 0xb8000;

    vidmem[0] = 'X';
    vidmem[1] = 0x02; /* cor do character: verde */
}
```

Código 3: conteúdo do arquivo *kernel.c*.

Para compilar este código, é necessário se compilar inicialmente o código `kernel.asm`, o qual representa o ponto de entrada de nosso kernel. Na sequência, também precisamos compilar o arquivo `kernel.c`, o qual será indicado para execução a partir do código `kernel.asm`. Após se compilar estes dois arquivos, iremos combiná-los, com o intuito de se construir um binário final para execução. Para compilar estes códigos, recomenda-se o uso do script apresentado no Código 4.

```
#!/bin/sh
nasm -f elf32 kernel.asm -o kasm.o
gcc -m32 -c -w kernel.c -o kc.o
ld -m elf_i386 -T link.ld -o kernel.bin kasm.o kc.o
```

Código 4: conteúdo do *shell script build.sh*.

Perceba no Código 4 que o arquivo `kernel.asm` é compilado utilizando-se o compilador Netwide Assembler (`nasm`), um *assembler* 80x86 portátil. Este código é gerado no formato Executable and Linking Format ou Extensible Linking Format (ELF) de 32 bits. Este padrão é flexível, extensível e multiplataforma, não ligado a qualquer unidade central de processamento (UCP ou CPU, derivado do termo em inglês) específica ou arquitetura de conjunto de instruções.

Isto o possibilitou ser adotado por muitos sistemas operacionais diferentes em muitas plataformas de hardware diferentes. Em seguida, o arquivo `kernel.c` é compilado com o `gcc`, utilizando a opção `-m32` para geração de código de 32 bits. Após a geração dos dois arquivos binários resultantes (`kasm.o` e `kc.o`), falta realizar a união destes, e claro, esta deve manter a ordem com o *entry point* sendo inserido primeiro (i.e. `kasm.o`).

Para realizar a união dos arquivos binários gerados, usaremos o *linker*. Assim sendo, é chegada a hora de especificar o arquivo `link.ld` (Código 5). O formato de saída após a junção de ambos os arquivos binários é ELF de 32 bits e a arquitetura é explicitamente especificada como i386. `ENTRY(start)` representa o *entry point* especificado em `kernel.asm` (lembra-se de `global start`?). Assim, o *kernel* irá iniciar a execução desta função e no ponto `call kmain` chama-se a função em `kernel.c`. após isso, o *kernel* irá pausar sua execução (*halt* em `hlt` em `kernel.asm`). As seções (SECTIONS) foram definidas anteriormente. A seção “.” representa a posição (i.e. endereço) a partir da qual o primeiro bit de código será armazenado. `.text`, aparece em `kernel.asm`. As demais seções serão explicadas à medida que forem necessárias (no momento serão ignoradas).

```
OUTPUT_FORMAT(elf32-i386)
ENTRY(start)
SECTIONS
{
    . = 0x100000;
    .text : { *(.text) }
    .data : { *(.data) }
    .bss  : { *(.bss) }
}
```

Código 5: conteúdo do arquivo `link.ld`.

A combinação dos arquivos binários é então realizada a partir da execução do *linker* (comando `ld`) na quarta linha do Código 4. Para tanto, o arquivo `link.ld` é necessário e a ordem na qual serão combinados os arquivos binários deve ser explicitada na linha de comando. A saída será o arquivo `kernel.bin`.

Para testar o *kernel* gerado, sugere-se o uso do software emulador `qemu`. O comando a ser executado é: `qemu-system-i386 --kernel kernel.bin` (em caso de variação deste, pesquise). Caso deseje, você pode inserir esta linha de código para execução automática do kernel (após sua geração) como parte do Código 4 ou ainda, você pode criar um *script* de execução separado. Para uma questão de completude, o próximo passo é a conversão do arquivo `kernel.bin` em um arquivo ISO, de modo a permitir a execução de nosso SO simples, por exemplo, no VirtualBox.

Para a geração do arquivo ISO, será necessário criar uma estrutura de diretórios, da seguinte forma: crie um diretório (ex. *simple*). Internamente ao diretório criado, crie um novo diretório chamado *boot* (ex. *simple/boot*) e por último, crie um diretório chamado *grub* (ex. *simple/boot/grub*). Dentro do diretório *grub*, crie um arquivo vazio chamado *grub.cfg*. Abra esse arquivo e insira o Código 6. Para que esse arquivo de configuração funcione corretamente, você deverá inserir o arquivo *kernel.bin* gerado anteriormente, no diretório *boot* criado.

```
set default = 0
set timeout = 0

menuentry "simpleOS" {
    set root='(hd96) '
    multiboot /boot/kernel.bin
}
```

Código 6: conteúdo do arquivo *grub.cfg*.

No arquivo *grub.cfg*, ajusta-se a execução deste SO como padrão (`set default = 0`), e o tempo aguardado para sua inicialização é ajustado em 0 segs (`set timeout = 0`). Além disso, é criada uma entrada de menu, para a escolha do SO, chamado de “*simpleOS*”. Nessa entrada, define-se o dispositivo de *boot* como sendo o CD-ROM (`set root='(hd96) '`), carregado a partir do arquivo *kernel.bin*, o qual está localizado na pasta *boot*.

Caso todos os passos tenham sido executados adequadamente, você agora fará a geração do ISO mediante o seguinte comando: “`grub-mkrescue -o simple.iso simple`”. Para testar a sua imagem ISO, crie uma máquina virtual no VirtualBox, com 64 MB de RAM, com descrições de “Other” para o tipo do SO e “Other/Unknown” para a versão (32 bits desconhecido). Para finalizar, configure esta máquina virtual para ser inicializada com um CD de boot, apontando para a imagem ISO criada (*simple.iso*). Para simplificar atividades futuras, você também pode inserir o código para geração do ISO como parte do Código 4.

Vale observar que, durante as atividades a serem realizadas, pode ser que não necessitemos realizar todos os passos apresentados no Código 4. Assim, é possível usar o símbolo de comentário (“#”) para evitar que determinadas partes do *script* sejam executadas. Outra dica, é mudar o local de saída do *kernel.bin*, de modo que este arquivo seja gerado diretamente dentro do diretório *boot* para a geração apropriada do arquivo ISO. Todas essas modificações são opcionais e ficam a critério do aluno.

TAREFA 2: ADICIONANDO FUNCIONALIDADES BÁSICAS AO KERNEL

Considerando a estrutura da prática de laboratório realizada na TAREFA 1, crie um subdiretório chamado “*include*”. Dentro desta pasta, serão criados quatro arquivos: *screen.h*,

`string.h`, `system.h` e `types.h`. O objetivo destes arquivos é criar e permitir a incorporação de novas funções ao kernel do SO. A partir de agora, as funções serão “habilitadas” (ex. uma função `print`) a partir da inclusão de bibliotecas mediante o uso da diretiva `#include` (como era de se esperar).

Faremos a criação da biblioteca de funções em ordem, começando pelo arquivo `types.h` (Código 7) o qual define os tipos de dados que serão usados nos demais arquivos de biblioteca. As duas primeiras linhas especificam a definição do arquivo de biblioteca. A inserção destas linhas (incluindo o `#endif` ao final) não é uma obrigatoriedade, mas faz parte das boas práticas de codificação de bibliotecas (ex. ao usar a ferramenta CodeBlocks, isso é padronizado ao se criar um arquivo de biblioteca). Na sequência, são definidos alguns tipos inteiros (de 8, 16, 32 e 64 bits) a partir de tipos primitivos de dados. Também foi criado um novo tipo chamado `string`, o qual representa uma cadeia de caracteres sem tamanho definido, a partir de um ponteiro para um tipo de dado primitivo caractere.

```
#ifndef TYPES_H
#define TYPES_H

typedef signed char int8;
typedef unsigned char uint8;

typedef signed short int16;
typedef unsigned short uint16;

typedef signed int int32;
typedef unsigned int uint32;

typedef signed long int64;
typedef unsigned long uint64;

typedef char* string;

#endif
```

Código 7: conteúdo do arquivo `types.h`.

Seguindo a sequência de definição para a biblioteca do SO, faz sentido especificar as funções para manipulação de *strings*. Assim sendo, especificaremos, a seguir o arquivo `string.h`. Por enquanto, este arquivo inclui apenas a função `uint16 strlen(string)`. A função depende dos tipos `uint16` e `string`, os quais foram definidos em `types.h`. A função descrita no Código 8 retorna um `uint16` (inteiro sem sinal de 16 bits) e recebe como parâmetro uma `string ch`. **Pergunta-se:** o que a função `uint16 strlen(string)` realiza?


```

#ifndef STRING_H
#define STRING_H

#include "types.h"
uint16 strlenh(string ch)
{
    uint16 i = 1;
    while(ch[i++]);
    return --i;
}
#endif

```

Código 8: conteúdo do arquivo *string.h*.

O próximo arquivo criado é o *system.h* (Código 9). O Código 9 apresenta duas funções uma retorna um valor `char` (`uint8`) e a outra não retorna valor (`void`). Para que possamos usar os tipos de dados que definimos anteriormente em *types.h*, incluímos este arquivo. A primeira função, `uint8 inportb (uint16)` recebe como parâmetro um inteiro de 16 bits e devolve um inteiro de 8 bits. O parâmetro de entrada é um endereço de porta (`_port`) de 16 bits. Internamente a função, a grosso modo, a variável `rv` será conectada a porta `_port` e o valor contido na porta será inserido nesta variável.

Talvez você se pergunte, por que precisamos deste código em Assembly. Por exemplo, se desejarmos controlar onde o cursor aparece em nossa tela, é necessário “falar” diretamente com as portas. Como existem dispositivos com os quais se deseja conectar (i.e. ler/escrever dados a partir deles), é necessário se usar portas. Na prática, o Código 9 é muito importante porque não é possível se falar com portas diretamente usando linguagem C e é por essa razão que foi criada uma função na linguagem Assembly.

Em alguns dispositivos, como a tela (*screen*), é possível efetuar a comunicação bastando-se escrever dados para uma área específica da memória (RAM). Esta continuará atualizando seus dados dependendo do que está guardado naquela área particular de memória. No caso do cursor, mouse e teclado, é necessário de fato se falar diretamente com as portas. Assim a função `uint8 inportb(uint16)` irá ler a partir da porta e a função `void outportb(uint 16, uint8)` irá escrever dados na porta. No caso da função `outportb(uint 16, uint8)` o primeiro argumento é a porta desejada e o segundo argumento são os dados que serão escritos para aquela porta particular.

Agora, trataremos dos aspectos mais técnicos do Código 9. Os prefixos e sufixos “`__`” (dois sublinhados) não chegam a ser necessários, entretanto, podem prevenir conflitos de nomes em programas. A notação apresentada é o que se chama de *extended asm* ou Assembly estendido. Esse é um código Assembly que pode ser usado diretamente dentro de código C para fins de acesso a registradores ou acesso a instruções de baixo nível.

```

#ifndef SYSTEM_H
#define SYSTEM_H
#include "types.h"

uint8 inportb (uint16 _port)
{
    uint8 rv;
    __asm__ __volatile__ ("inb %1, %0"
                          : "=a" (rv)          /* operando de saída */
                          : "dN" (_port)); /* operando de entrada */
    return rv;
}

void outportb (uint16 _port, uint8 _data)
{
    __asm__ __volatile__ ("outb %1, %0"
                          :                    /* operando de saída */
                          : "dN" (_port), "a" (_data)); /* operando de entrada */
}

#endif

```

Código 9: conteúdo do arquivo *system.h*.

O código *assembly* estendido permite a especificação de registradores de entrada, de saída e *clobbered* (podem ser sobrescritos durante a execução). No Código 9, este foi indentado de modo a demonstrar cada uma das linhas de entrada/saída/*clobbered* (não usado). A primeira linha que começa com um ‘:’ especifica os operandos de saída, a segunda linha indica os operandos de entrada e, caso existisse, a terceira linha especificaria os operandos *clobbered*. ‘a’ é um exemplo de constante¹. Constantes de saída devem possuir como prefixo um ‘=’, como é o caso do “=a” (= é um modificador indicando *write only*). Constantes de entrada e saída devem possuir seu argumento em C correspondente entre parêntesis (não vale para a linha de parâmetro *clobbered*). Note o uso de “0” e “1”. Estes são usados para assegurar que quando uma mesma variável é indicada em mais de um lugar no *asm* estendido, esta variável é mapeada apenas para um registrador. Em caso de mais de uma referência a mesma variável, o compilador pode ou não colocá-la no mesmo registrador já atribuído.

As funções `inb` e `outb` são funções `__asm__` para a realização de acesso a entrada e saída (I/O)². No Código 9, na linha de código `__asm__` estendido especificando `inportb`, é possível verificar a leitura do valor partir de `_port` e a atribuição deste a `rv`. A diretiva `__asm__` `__volatile__` foi usada devido aos mecanismos de otimização do compilador. Graças a essas otimizações, o código pode ou não aparecer na localidade especificada pelo programador. Para evitar isso, usa-se essa diretiva, ao invés de simplesmente `__asm__`.

1 Conforme ilustrado em: <http://asm.sourceforge.net/articles/rmiyagi-inline-asm.txt>.

2 Conforme ilustrado em: https://wiki.osdev.org/Inline_Assembly/Examples e <https://stackoverflow.com/questions/8960620/low-level-i-o-access-using-outb-and-inb>.


```

#ifndef SCREEN_H
#define SCREEN_H
#include "types.h"
#include "system.h"
#include "string.h"

int32 cursorX = 0, cursorY = 0;
const uint8 sw = 80, sh = 25, sd = 2;

void clearLine(uint8 from, uint8 to)
{
    uint16 i = sw * from * sd;
    string vidmem = (string)0xb8000;
    for (i; i < (sw * (to + 1) * sd); i++)
    {
        vidmem[i] = 0x0;
    }
}

void updateCursor()
{
    unsigned temp;
    temp = cursorY * sw + cursorX;

    outportb(0x3D4, 14);
    outportb(0x3D5, temp >> 8);
    outportb(0x3D4, 15);
    outportb(0x3D5, temp);
}

void clearScreen()
{
    clearLine(0, sh-1);
    cursorX = 0;
    cursorY = 0;
    updateCursor();
}

void scrollUp(uint8 lineNumber)
{
    string vidmem = (string)0xb8000;
    uint16 i = 0;

    for (i; i < sw * (sh - 1) * sd; i++)
    {
        vidmem[i] = vidmem[i + sw * sd * lineNumber];
    }

    clearLine(sh - 1 - lineNumber, sh - 1);

    if ((cursorY - lineNumber) < 0)
    {
        cursorY = 0;
        cursorX = 0;
    }
    else
    {
        cursorY -= lineNumber;
    }
    updateCursor();
}

void newLineCheck()

```

```

{
    if(cursorY >= sh-1)
    {
        scrollUp(1);
    }
}

void printch(char c)
{
    string vidmem = (string)0xb8000;

    switch(c)
    {
        case (0x08):
            if(cursorX > 0)
            {
                cursorX--;
                vidmem[(cursorY * sw + cursorX)*sd] = 0x00;
            }
        case (0x09): // Representa a tecla TAB
            cursorX = (cursorX + 8) & ~(8 - 1);
            break;
        case ('\r'):
            cursorX = 0;
            break;
        case ('\n'):
            cursorX = 0;
            cursorY++;
            break;
        default:
            vidmem[((cursorY * sw + cursorX)) * sd] = c;
            vidmem[((cursorY * sw + cursorX)) * sd + 1] = 0x0F;
            cursorX++;
            break;
    }

    if (cursorX >= sw)
    {
        cursorX = 0;
        cursorY++;
    }
    newLineCheck();
    updateCursor();
}

void print(string ch)
{
    uint16 i = 0;

    for(i; i < strlen(ch); i++)
    {
        printch(ch[i]);
    }
}
#endif

```

Código 10: conteúdo do arquivo *screen.h*.

A função `void clearLine(uint8, uint8)`, limpa a tela partir de uma linha específica até outra linha específica. Esta função utiliza uma variável auxiliar `i`, que define o ponto de partida em função de largura (i.e. colunas) e profundidade, para a posição pretendida (`from`). A variável `vidmem` é uma `string` a qual é inicializada com o valor `0xb8000` (endereço da primeira posição válida na tela). Como se trata de um endereço numérico, faz-se um *casting* por segurança. Na sequência, todas as posições até `to+1` (lembre-se que começamos em 0) são limpas

em valor e cor (valor 0 e cor preta, também 0).

A função `void updateCursor(void)`, envia a localização do cursor para uma porta específica (0x3D4 e 0x3D5). Estas portas são padronizadas para a arquitetura x86 e controlam a posição do cursor na tela em modo VGA (modo usando 80 colunas x 25 linhas x 16 cores). Assim, para mudar a posição do cursor, basta atualizar os valores de `x` e `y` e então chamar a função `void updateCursor(void)`. **Neste momento, pede-se ao aluno que salve seu trabalho e tente utilizar a função `updateCursor()` no arquivo `kernel.c`. Compile e execute o seu trabalho usando o `qEmu` e verifique o resultado. Utilize os valores de `cursorX = 20` e `cursorY=15` e teste.** Explicando esta função um pouco melhor, a variável `temp` recebe a posição em função dos eixos `x` e `y`. A função `outportb()` já foi explicada, e em `outportb(0x3D4, 14)`, seleciona a localização do cursor no registrador de controle (Control Register ou CRT). Em `outportb(0x3D5, temp >> 8)`, ela envia o byte de alta no barramento. Em `outportb(0x3D4, 15)`, seleciona o envio do byte de baixa no CRT e em `outportb(0x3D5, temp)` envia o byte de baixa da localização do cursor no barramento.

A função `void clearScreen(void)` faz a limpeza da tela da primeira a última linha e atualiza a posição do cursor nos eixos `x` e `y` para 0.

A função `void scrollUp(uint8)` desloca o texto que aparece na tela `lineNumber` linhas.

A função `void printch(char)` imprime um caractere na tela e a função `void print(string)` imprime uma `string` na tela.

A função `void newLineCheck(void)` é uma função de verificação para confirmar se alcançamos a última linha na tela, tomando as medidas cabíveis.

Como parte de sua última atividade, **insira comentários detalhados para as funções `void clearScreen(void)`, `void scrollUp(uint8)`, `void printch(char)`, `void print(string)` e `void newLineCheck()`**. Utilize estas funções no contexto de `kernel.c` demonstrando o funcionamento de cada uma delas.

Como observação final, vale destacar que todas as funções declaradas são consideradas "estáticas". Estática aqui não deve ser interpretado como a maneira como se codificam funções em linguagens de alto nível. "Estática" aqui, significa dizer que, da maneira como as funções foram codificadas, se qualquer uma delas for chamada dentro do *kernel* dez vezes, ela será escrita e reescrita internamente ao *kernel* dez vezes. Isso tem o potencial para gerar uma grande quantidade de código repetido. O que irá refletir também no tamanho do executável final. Para otimizar isso, recomenda-se declarar todas as funções com "external" e implementá-las em um código ".c" separado (gerando um código objeto próprio), ao invés de inseri-las dentro do arquivo ".h". A vantagem de se organizar o código desta forma é que, toda vez que a função for chamada, não é necessário reescrevê-la. Basicamente, o ambiente de tempo de execução irá procurá-la no lugar onde ela está definida (i.e. seus binários), e a executa sem repetição de código. Ao adicionar tal função ".c", será necessário introduzir sua compilação no arquivo `build.sh` de maneira análoga aos demais códigos que lá estão.

REFERÊNCIAS

DUARTE, G. How Computers Boot Up. Many But Fine – Tech and science for curious people. Available: <https://manybutfinite.com/post/how-computers-boot-up/> (Access: 31/08/2018).

@0xAX. Linux-insides - A book-in-progress about the linux kernel and its insides. Available: <https://0xax.gitbooks.io/linux-insides/content/Bootting/linux-bootstrap-1.html> (Access: 31/08/2018).